

Senior Full Stack Engineer - Take home assessment

Context

As a Senior Full Stack Engineer, we expect you to be able to solve ambiguous problems with effective and well thought out solutions, even if you need to solve the problem with new or unfamiliar technologies.

Therefore this is an open-ended challenge to test your technical depth, technical experience, problem-solving skills, adaptability, and effective communication.

You should spend at least 1-2 hours working on this. You can spend more time on it if needed but it is **optional** and you will not be penalised for it, so please plan and prioritise what will go into the MVP carefully first.

The goal is not for you to build a full production ready product, the goal is to test if you have the ability to do so by looking at the first steps that you made for the project and your ability to think and reason about the path from zero to production and to legacy maintenance.

Build this product using your own understanding and interpretation of what a Senior Engineer is expected to deliver within the time limit.

Goal

Build a real time messaging web app (full stack) for 1 to 1 private messaging between users, sort of like a simple web based whatsapp or telegram app.

Hard requirements (What your MVP must achieve at a minimum)

1. The MVP must be able to work/run.
2. The MVP must be able to work/run in a docker container or docker compose / swarm setup.
3. It should be a full stack web app that does not rely solely on frontend based peer-to-peer technologies, i.e. there should be some sort of backend for this.
4. It must be real time and work across devices (simulated by opening the web app in 2 different browser windows and messaging each other)
5. The core feature must be implemented by yourself without using any existing frameworks/libraries that does the whole thing. Meaning you can't just be building a UI wrapper on top of a framework that does this. (More details below)
6. Have some level of code documentation, focusing on the context (why) rather than what the code does, e.g. comments like "this has to use X because of this particular edge case which might cause race conditions" rather than "the next line uses X".

Constraints (What are you not allowed to do)

1. You are **cannot** use any existing frameworks/libraries that implement the core features fully, for example, you cannot use a framework that has built in peer to peer messaging support built in and you just build a UI wrapper around it.
 1. However you can use frameworks/libraries for none core features, i.e. you can use UI frameworks like React/Vue, an existing database ORM, session cookie management library

and etc...

2. You cannot integrate with external SaaS APIs / services, it must be able to run completely locally on your device.

What are we looking for?

1. Your ability to foresee potential use cases and obstacles and plan for them
 1. Your MVP does not need to handle every single edge case, however you must at least know about the potential obstacles / edge case and be able to list them out either in your code comments, write it down or answer directly in the follow up interview.
 2. Here is a chance for your experience to shine, we are not looking for just default text book answers like "if we dont encrypt the client-server communication with HTTPS it can be hacked via MITM attacks", instead what we are looking for is more like "if we did X using the Y method, although it will work, once there is more users it can easily cause race conditions because of Z, and i've experienced issues like this while i was working on this other project previously, and here is how i solved it the last time..."
2. Your ability to work on technical features at least one level deeper than the product integration level
 1. Meaning instead of just simply gluing libraries/frameworks together to get a product working, we are looking to see if you are able to build with lower level technology (standard library or on top of lower level primitives) if you didnt have access to existing libraries/frameworks.
3. Your ability to communicate your thoughts clearly and effectively to both technical and non-technical audiences.

Additional notes

1. You can write your MVP in any language you like.
2. Be prepared to make changes, implement new features, answer more technical questions regarding your technical decisions on the spot during the follow up interview.
3. You can use AI to assist you in building the project, however you will not be able to use any AI during the follow up interview, you will have to make changes manually on the spot.